Lab Sheet 03 · Part B Answers

HDIT 12103 Website Design and Development · Validation

Task B1: Let the browser check

Step 2. What the browser does (message wording differs between browsers; these are Chrome-style):

Try	What the browser does
Full name: Ab	Blocks the submit and shows a bubble: please lengthen this text to 3 characters or more (you are using 2).
Student ID left empty	Blocks the submit: "Please fill out this field." (required)
Email: rizwanmuzawir@gmail	Accepts it. No bubble for the email field, because type="email" allows an address with no dot after the @.
Phone: 12345	Blocks the submit: "Please match the requested format." (pattern)
Guests: 5	Blocks the submit: "Value must be less than or equal to 3." (max)

Step 3. The browser accepts rizwanmuzawir@gmail because the HTML email type only needs something@something. On a campus registration list this lets through addresses that are not real or deliverable, so confirmation emails would bounce and the organisers could not contact the student. This is one reason to add our own regex check in JavaScript.

Task B2.1: Predict, then run

#	Code	Your prediction	Actual
1	`/^07\d{8}$/.test("0771234567")`	true	true
2	`/^07\d{8}$/.test("077 123 4567")`	false (has spaces)	false
3	`/^07\d{8}$/.test("771234567")`	false (no 07, only 9 digits)	false
4	`/^07\d{8}$/.test("07712345678")`	false (11 digits, too long)	false
5	`/^HD\d-\d{3}$/.test("HD3-017")`	true	true
6	`/^HD\d-\d{3}$/.test("hd3-017")`	false (case-sensitive)	false
7	`/^HD\d-\d{3}$/.test("HD3-17")`	false (only 2 digits)	false
8	`/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test("muzawir@icst.lk")`	true	true
9	`/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test("muzawir@icst")`	false (no dot after @)	false
10	`/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test("muzawir rizwan@icst.lk")`	false (contains a space)	false

Remove ^ and $ from row 4's pattern: `/07\d{8}/.test("07712345678")` changes from **false** to **true**. Without ^ and $ the regex only looks for 07 followed by 8 digits *anywhere* in the text, so it finds a match in the first 10 characters and ignores the extra digit. ^ and $ force the whole value to match, not just part of it.

Same checks with my own details:

Code	Result	Why
`/^07\d{8}$/.test("0720470471")`	true	My mobile: 07 then 8 digits
`/^HD\d-\d{3}$/.test("26015")`	false	No HD prefix and no dash, so my Part A student ID would fail here

Code	Result	Why
`/^HD\d-\d{3}$/.test("HD3-017")`	true	The sheet's example format passes
`/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test("rizwanmuzawir@gmail.com")`	true	My email has a dot after the @
`/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test("rizwanmuzawir@gmail")`	false	No dot after the @

Task B2.2: Validate every field

The code is in `js/part-b.js`. Test it on `part-b.html`, which loads `part-b.js` and has `novalidate` on the form tag. Expected test results:

Test	Expected
Submit the empty form	Five red messages (name, student ID, email, phone, workshop); guests is fine at 0. The cursor jumps to Full name.
Valid except phone "072 047 0471"	Only the phone message shows. The cursor jumps to the phone box.
Guests 2.5, then -1, then 3	Error, error, no error.
Everything valid (Muzawir Rizwan, HD3-017, rizwanmuzawir@gmail.com, 0720470471, Responsive web design, 2 guests)	Registration complete! and the form empties.

Task B3: Live feedback

Do this	Result
Type riz in the email box	No error yet: the field has not been left, so the blur event has not fired.
Press Tab to leave the box	Enter an email like name@example.com.
Click back and finish typing wanmuzawir@gmail.com	The error disappears while you type, because the input event now rechecks the field.

B3.2 (Challenge): showError also toggles a `valid` class, so a touched field with no error gets a green border. Add this CSS: `.valid { border: 2px solid #2e7d32; }`

Task B4: Check it without a mouse

1. Each input now has `aria-describedby="…-error"`, pointing to its error span (visible in the Elements panel).

2. Anything you could not do: *nothing; every field can be reached with Tab and Shift+Tab, the radio buttons and select work with the arrow keys, Space ticks the checkbox and Enter submits.* (Confirm this in your own run and write down anything different.)

3. With a wrong phone number the cursor goes to the **phone box**, because the submit handler calls `firstBad.focus()`. This helps a keyboard user because they do not have to Tab through the whole form to find the problem, and a screen reader then reads the field together with its error message.

Task B5: Break your own validation

3. Yes, the invalid email was sent: the address bar shows `?fullName=&…&email=not-an-email&…`. Our validation did not run because `form.submit()` sends the form directly and does **not** fire the `submit` event, so our listener never ran (it also skips the browser's own checks).

4. Two other ways to get past the checks:
• Turn off JavaScript in the browser, or edit the page in DevTools (remove `novalidate`, `pattern` or change the values) before submitting.
• Skip the page entirely and send the request with a tool such as curl or Postman. This is why the server must validate every value again.